Chapter 17. Extending Kubernetes

From the beginning, it was clear that Kubernetes was going to be more than its core set of APIs; once an application is orchestrated within the cluster, there are countless other useful tools and utilities that can be represented and deployed as API objects in the Kubernetes cluster. The challenge was how to embrace this explosion of objects and use cases without having an API that sprawled without bound.

To resolve this tension between extended use cases and API sprawl, significant effort was put into making the Kubernetes API extensible. This extensibility meant that cluster operators could customize their clusters with the additional components that suited their needs. This extensibility enables people to augment their clusters themselves, consume community-developed cluster add-ons, and even develop extensions that are bundled and sold in an ecosystem of cluster plug-ins. Extensibility has also given rise to whole new patterns of managing systems, such as the operator pattern.

Regardless of whether you are building your own extensions or consuming operators from the ecosystem, understanding how the Kubernetes API server is extended and how extensions can be built and delivered is a key component to unlocking the complete power of Kubernetes and its ecosystem. As more and more advanced tools and platforms are built on top of Kubernetes using these extensibility mechanisms, a working knowledge of how they operate is critical to understanding how to build applications in a modern Kubernetes cluster.

What It Means to Extend Kubernetes

In general, extensions to the Kubernetes API server either add new functionality to a cluster or limit and tweak the ways that users can interact with their clusters. There is a rich ecosystem of plug-ins that cluster administrators can use to add services and capabilities to their clusters. It's worth noting that extending the cluster is a very high-privilege thing to do. It is not a capability that should be extended to arbitrary users or arbitrary code because cluster administrator privileges are required to extend a cluster. Even cluster administrators should be careful and use diligence when installing third-party tools. Some extensions, like admission controllers, can be used to view all objects being

created in the cluster, and could easily be used as a vector to steal Secrets or run malicious code. Additionally, extending a cluster makes it different than stock Kubernetes. When running on multiple clusters, it is very valuable to build tooling to maintain consistency of experience across the clusters, and this includes the extensions that are installed.

Points of Extensibility

There are many ways to extend Kubernetes, from CustomResourceDefinitions to Container Network Interface plug-ins. This chapter is going to focus on extending the API server by adding new resource types or admission controllers to API requests. We will not cover CNI/CSI/CRI (Container Network Interface/Container Storage Interface/Container Runtime Interface) extensions, as they are more commonly used by Kubernetes cluster providers rather than by the Kubernetes end users, for whom this book was written.

In addition to admission controllers and API extensions, there are actually a number of ways to "extend" your cluster without ever modifying the API server at all. These include DaemonSets that install automatic logging and monitoring, tools that scan your services for cross-site scripting (XSS) vulnerabilities, and more. Before embarking on extending your cluster yourself, however, it's worth considering the landscape of things that are possible within the confines of the existing Kubernetes APIs.

To understand the role of admission controllers and CustomResourceDefinitions, it helps to review the flow of requests through the Kubernetes API server, shown in Figure 17-1.

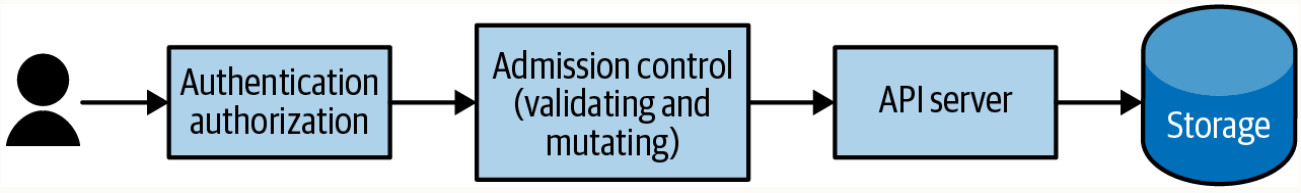


Figure 17-1. API server request flow

Admission controllers are called prior to the API object being written into the backing storage. Admission controllers can reject or modify API requests. Several admission controllers are built into the Kubernetes API server; for example, the limit range admission controller that sets default limits for Pods without them. Many other systems use custom admission controllers to auto-inject sidecar containers into all Pods created on the system to enable "auto-magic" experiences.

The other form of extension, which can also be used in conjunction with admission controllers, is *custom resources*. With custom resources, whole new API objects are added to the Kubernetes API surface area. These new API objects can be added to namespaces, are subject to RBAC, and can be accessed with existing tools like `kubectl` as well as via the Kubernetes API.

The following sections describe these Kubernetes extension points in greater detail and give both use cases and hands-on examples of how to extend your cluster.

The first thing to do to create a custom resource is to create a CustomResourceDefinition. This object is actually a meta-resource; that is, a resource that is the definition of another resource.

As a concrete example, consider defining a new resource to represent load tests in your cluster. When a new LoadTest resource is created, a load test is spun up in your Kubernetes cluster and drives traffic to a service.

The first step in creating this new resource is defining it through a CustomResourceDefinition. An example definition looks as follows:

```
apiVersion: apiextensions.k8s.io/v1beta1
kind: CustomResourceDefinition
metadata:
  name: loadtests.beta.kuar.com
spec:
  group: beta.kuar.com
  versions:
    - name: v1
      served: true
      storage: true
  scope: Namespaced
  names:
    plural: loadtests
    singular: loadtest
    kind: LoadTest
    shortNames:
    - lt
```

You can see that this is a Kubernetes object like any other. It has a `metadata` sub-object, and within that sub-object, the resource is named. However, in the case of custom resources, the name is special. It has to be the format `<resource-plural>.<api-group>` to ensure that each resource definition is unique in the cluster, because the name of each

CustomResourceDefinition has to match this pattern, and no two objects in the cluster can have the same name. We are thus guaranteed that no two CustomResourceDefinitions define the same resource.

In addition to metadata, the CustomResourceDefinition has a `spec` sub-object. This is where the resource itself is defined. In that `spec` object, there is an `apigroup` field that supplies the API group for the resource. As mentioned previously, it must match the suffix of the CustomResourceDefinition's name. Additionally, there is a list of versions for the resource, which includes the name of the version (e.g., `v1`, `v2`, etc.), as well as fields that indicate if that version is served by the API server and which version is used for storing data in the backing storage for the API server. The `storage` field must be true for only a single version for the resource. There is also a `scope` field to indicate whether the resource is namespaced (the default is namespaced), and a `names` field that allows for the definition of the singular, plural, and `kind` values for the resource. It also allows the definition of convenience "short names" for the resource for use in `kubectl` and elsewhere.

Given this definition, you can create the resource in the Kubernetes API server. But first, to show the true nature of dynamic resource types, try to list our `loadtests` resource using `kubectl`:

```
$ kubectl get loadtests
```

You'll see that there is no such resource currently defined. Now use *loadtest-resource.yaml* to create this resource:

```
$ kubectl create -f loadtest-resource.yaml
```

Then get the `loadtests` resource again:

```
$ kubectl get loadtests
```

This time you'll see that there is a LoadTest resource type defined, though there are still no instances of this resource type. Let's change that by creating a new LoadTest resource.

As with all built-in Kubernetes API objects, you can use YAML or JSON to define a custom resource (in this case our LoadTest). See the following definition:

```
apiVersion: beta.kuar.com/v1
kind: LoadTest
metadata:
  name: my-loadtest
spec:
  service: my-service
  scheme: https
  requestsPerSecond: 1000
  paths:
  - /index.html
  - /login.html
  - /shares/my-shares/
```

One thing you'll note is that we never defined the schema for the custom resource in the CustomResourceDefinition. It actually is possible to provide an OpenAPI specification (known previously as Swagger) for a custom resource, but this complexity is generally not worth it for simple resource types. If you do want to perform validation, you can register a validating admission controller, as described in the following sections.

You can now use this *loadtest.yaml* file to create a resource just like you would with any built-in type:

```
$ kubectl create -f loadtest.yaml
```

Now when you list the `loadtests` resource, you'll see your newly created resource:

```
$ kubectl get loadtests
```

This may be exciting, but it doesn't really do anything yet. Sure, you can use this simple CRUD (Create/Read/Update/Delete) API to manipulate the data for LoadTest objects, but no actual load tests are created in response to this new API we defined because there is no controller present in the cluster to react and take action when a LoadTest object is defined. The LoadTest custom resource is only half of the infrastructure needed to add LoadTests to our cluster. The other half is a piece of code that will continuously monitor the custom resources and create, modify, or delete LoadTests as necessary to implement the API.

Just like the user of the API, the controller interacts with the API server to list LoadTests and watches for any changes that might occur. This interaction between controller and

API server is shown in Figure 17-2.

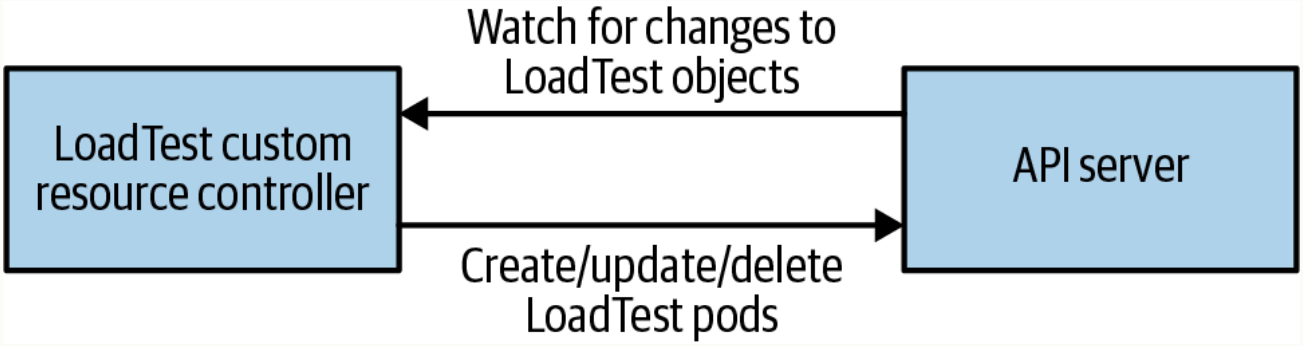


Figure 17-2. CustomResourceDefinition interactions

The code for such a controller can range from simple to complex. The simplest controllers run a `for` loop and repeatedly poll for new custom objects, and then take actions to create or delete the resources that implement those custom objects (e.g., the LoadTest worker Pods).

However, this polling-based approach is inefficient: the period of the polling loop adds unnecessary latency, and the overhead of polling may add unnecessary load on the API server. A more efficient approach is to use the watch API on the API server, which provides a stream of updates when they occur, eliminating both the latency and overhead of polling. However, using this API correctly in a bug-free way is complicated. As a result, if you want to use watches, it is highly recommended that you use a well-supported mechanism such as the `Informer` pattern exposed in the *client-go* library.

Now that we have created a custom resource and implemented it via a controller, we have the basic functionality of a new resource in our cluster. However, many parts of what it means to be a well-functioning resource are missing. The two most important are validation and defaulting. *Validation* is the process of ensuring that LoadTest objects sent to the API server are well formed and can be used to create load tests, while *defaulting* makes it easier for people to use our resources by providing automatic, commonly used values by default. We'll now cover adding these capabilities to our custom resource.

As mentioned earlier, one option for adding validation is via an OpenAPI specification for our objects. This can be useful for basic validation of the presence of required fields or the absence of unknown fields. A complete OpenAPI tutorial is beyond the scope of this book, but there are lots of resources online, including the complete Kubernetes API specification.

Generally speaking, an API schema is actually insufficient for validation of API objects. For example, in our `loadtests` example, we may want to validate that the LoadTest ob-

ject has a valid scheme (e.g., *http* or *https*) or that `requestsPerSecond` is a nonzero positive number.

To accomplish this, we will use a validating admission controller. As discussed previously, admission controllers intercept requests to the API server before they are processed and can reject or modify the requests in flight. Admission controllers can be added to a cluster via the dynamic admission control system. A dynamic admission controller is a simple HTTP application. The API server connects to the admission controller via either a Kubernetes Service object or an arbitrary URL. This means that admission controllers can optionally run outside of the cluster—for example, in a cloud provider's Function-as-a-Service offering, like Azure Functions or AWS Lambda.

To install our validating admission controller, we need to specify it as a Kubernetes ValidatingWebhookConfiguration. This object specifies the endpoint where the admission controller runs, as well as the resource (in this case LoadTest) and the action (in this case `CREATE`) where the admission controller should be run. You can see the full definition for the validating admission controller in the following code:

```
apiVersion: admissionregistration.k8s.io/v1beta1
kind: ValidatingWebhookConfiguration
metadata:
  name: kuar-validator
webhooks:
- name: validator.kuar.com
  rules:
  - apiGroups:
    - "beta.kuar.com"
    apiVersions:
    - v1
    operations:
    - CREATE
    resources:
    - loadtests
  clientConfig:
    # Substitute the appropriate IP address for your webhook
    url: https://192.168.1.233:8080
    # This should be the base64-encoded CA certificate for your cluster,
    # you can find it in your ${KUBECONFIG} file
    caBundle: REPLACEME
```

Fortunately for security, but unfortunately for complexity, webhooks that are accessed by the Kubernetes API server can only be accessed via HTTPS. So we need to generate a

certificate to serve the webhook. The easiest way to do this is to use the cluster's ability to generate new certificates using its own certificate authority (CA).

First, we need a private key and a certificate signing request (CSR). Here's a simple Go program that generates these:

```
package main

import (
	"crypto/rand"
	"crypto/rsa"
	"crypto/x509"
	"crypto/x509/pkix"
	"encoding/asn1"
	"encoding/pem"
	"net/url"
	"os"
)

func main() {
	host := os.Args[1]
	name := "server"

	key, err := rsa.GenerateKey(rand.Reader, 1024)
	if err != nil {
		panic(err)
	}
	keyDer := x509.MarshalPKCS1PrivateKey(key)
	keyBlock := pem.Block{
		Type:  "RSA PRIVATE KEY",
		Bytes: keyDer,
	}
	keyFile, err := os.Create(name + ".key")
	if err != nil {
		panic(err)
	}
	pem.Encode(keyFile, &keyBlock)
	keyFile.Close()

	commonName := "myuser"
	emailAddress := "someone@myco.com"

	org := "My Co, Inc."
	orgUnit := "Widget Farmers"
	city := "Seattle"
```

```
        state := "WA"
        country := "US"

        subject := pkix.Name{
                CommonName:        commonName,
                Country:           []string{country},
                Locality:          []string{city},
                Organization:      []string{org},
                OrganizationalUnit: []string{orgUnit},
                Province:          []string{state},
        }

        uri, err := url.ParseRequestURI(host)
        if err != nil {
                panic(err)
        }

        asn1, err := asn1.Marshal(subject.ToRDNSequence())
        if err != nil {
                panic(err)
        }
        csr := x509.CertificateRequest{
                RawSubject:        asn1,
                EmailAddresses:    []string{emailAddress},
                SignatureAlgorithm: x509.SHA256WithRSA,
                URIs:              []*url.URL{uri},
        }

        bytes, err := x509.CreateCertificateRequest(rand.Reader, &csr, key)
        if err != nil {
                panic(err)
        }
        csrFile, err := os.Create(name + ".csr")
        if err != nil {
                panic(err)
        }

        pem.Encode(csrFile, &pem.Block{Type: "CERTIFICATE REQUEST", Bytes: bytes})
        csrFile.Close()
}
```

You can run this program with:

```
$ go run csr-gen.go <URL-for-webhook>
```

and it will generate two files, *server.csr* and *server-key.pem*.

You can then create a certificate signing request for the Kubernetes API server using the following YAML:

```
apiVersion: certificates.k8s.io/v1beta1
kind: CertificateSigningRequest
metadata:
  name: validating-controller.default
spec:
  groups:
  - system:authenticated
  request: REPLACEME
  usages:
  usages:
  - digital signature
  - key encipherment
  - key agreement
  - server auth
```

You will notice for the `request` field the value is `REPLACEME`; this needs to be replaced with the base64-encoded certificate signing request we produced in the preceding code:

```
$ perl -pi -e s/REPLACEME/$(base64 server.csr | tr -d '\n')/ \
admission-controller-csr.yaml
```

Now that your certificate signing request is ready, you can send it to the API server to get the certificate:

```
$ kubectl create -f admission-controller-csr.yaml
```

Next, you need to approve that request:

```
$ kubectl certificate approve validating-controller.default
```

Once approved, you can download the new certificate:

```
$ kubectl get csr validating-controller.default -o json | \
```

```
jq -r .status.certificate | base64 -d > server.crt
```

With the certificate, you are finally ready to create an SSL-based admission controller (phew!). When the admission controller code receives a request, it contains an object of type `AdmissionReview`, which contains metadata about the request as well as the body of the request itself. In our validating admission controller, we have only registered for a single resource type and a single action (`CREATE`), so we don't need to examine the request metadata. Instead, we dive directly into the resource itself and validate that `requestsPerSecond` is positive and the URL scheme is valid. If they aren't, we return a JSON body disallowing the request.

Implementing an admission controller to provide defaulting is similar to the steps just described, but instead of using a ValidatingWebhookConfiguration, you use a MutatingWebhookConfiguration, and you need to provide a JSON Patch object to mutate the request object before it is stored.

Here's a TypeScript snippet that you can add to your validating admission controller to add defaulting. If the `paths` field in the `loadtest` is of length zero, add a single path for `/index.html`:

```
    if (needsPatch(loadtest)) {
      const patch = [
        { 'op': 'add', 'path': '/spec/paths', 'value': ['/index.html'] },
      ]
      response['patch'] = Buffer.from(JSON.stringify(patch))
        .toString('base64');
      response['patchType'] = 'JSONPatch';
    }
```

You can then register this webhook as a MutatingWebhookConfiguration by simply changing the `kind` field in the YAML object and saving the file as *mutating-controller.yaml*. Then create the controller by running:

```
$ kubectl create -f mutating-controller.yaml
```

At this point, you've seen a complete example of how to extend the Kubernetes API server using custom resources and admission controllers. The following section describes some general patterns for various extensions.

Patterns for Custom Resources

Not all custom resources are identical. There are a variety of reasons for extending the Kubernetes API surface area, and the following sections discuss some general patterns you may want to consider.

Just Data

The easiest pattern for API extension is the notion of "just data." In this pattern, you are simply using the API server for storage and retrieval of information for your application. It is important to note that you should *not* use the Kubernetes API server for application data storage. The Kubernetes API server is not designed to be a key/value store for your app; instead, API extensions should be control or configuration objects that help you manage the deployment or runtime of your application. An example use case for the "just data" pattern might be configuration for canary deployments of your application—for example, directing 10% of all traffic to an experimental backend. While in theory such configuration information could also be stored in a ConfigMap, ConfigMaps are essentially untyped, and sometimes using a more strongly typed API extension object provides clarity and ease of use.

Extensions that are just data don't need a corresponding controller to activate them, but they may have validating or mutating admission controllers to ensure that they are well formed. For example, in the canary use case, a validating controller might ensure that all percentages in the canary object sum to 100%.

Compilers

A slightly more complicated pattern is the "compiler" or "abstraction" pattern. In this pattern, the API extension object represents a higher-level abstraction that is "compiled" into a combination of lower-level Kubernetes objects. The LoadTest extension in the previous example is an example of this compiler abstraction pattern. A user consumes the extension as a high-level concept, in this case a `loadtest`, but it comes into being by being deployed as a collection of Kubernetes Pods and services. To achieve this, a compiled abstraction requires an API controller to be running somewhere in the cluster to watch the current LoadTests and create the "compiled" representation (and likewise delete representations that no longer exist). In contrast to the operator pattern described next, however, there is no online health maintenance for compiled abstractions; it is delegated down to the lower-level objects (e.g., Pods).

Operators

While compiler extensions provide easy-to-use abstractions, extensions that use the "operator" pattern provide online, proactive management of the resources created by the extensions. These extensions likely provide a higher-level abstraction (for example, a database) that is compiled down to a lower-level representation, but they also provide online functionality, such as snapshot backups of the database or upgrade notifications when a new version of the software is available. To achieve this, the controller not only monitors the extension API to add or remove things as necessary, but also monitors the running state of the application supplied by the extension (e.g., a database) and takes actions to remediate unhealthy databases, take snapshots, or restore from a snapshot if a failure occurs.

Operators are the most complicated pattern for API extension of Kubernetes, but they are also the most powerful, enabling users to get easy access to "self-driving" abstractions that are responsible not just for deployment, but also health checking and repair.

Getting Started

Getting started extending the Kubernetes API can be a daunting and exhausting experience. Fortunately, there is a great deal of code to help you out. The Kubebuilder project contains a library of code intended to help you easily build reliable Kubernetes API extensions. It's a great resource to help you bootstrap your extension.

Summary

One of the great "superpowers" of Kubernetes is its ecosystem, and one of the most significant things powering this ecosystem is the extensibility of the Kubernetes API. Whether you're designing your own extensions to customize your cluster or consuming off-the-shelf extensions as utilities, cluster services, or operators, API extensions are the key to making your cluster your own and building the right environment for the rapid development of reliable applications.

‹ 16. Integrating Storage Solutions and Kubernetes

18. Accessing Kubernetes from Common Programming Languages ›